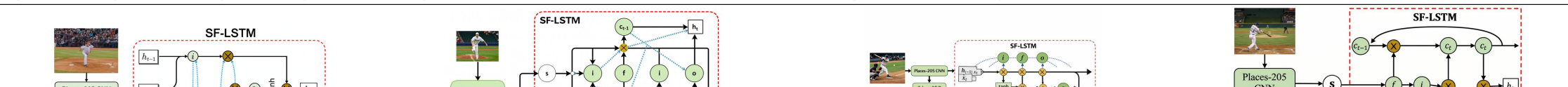
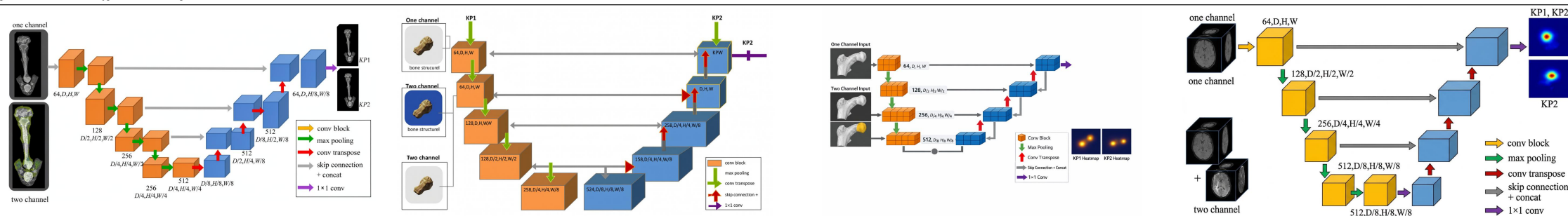


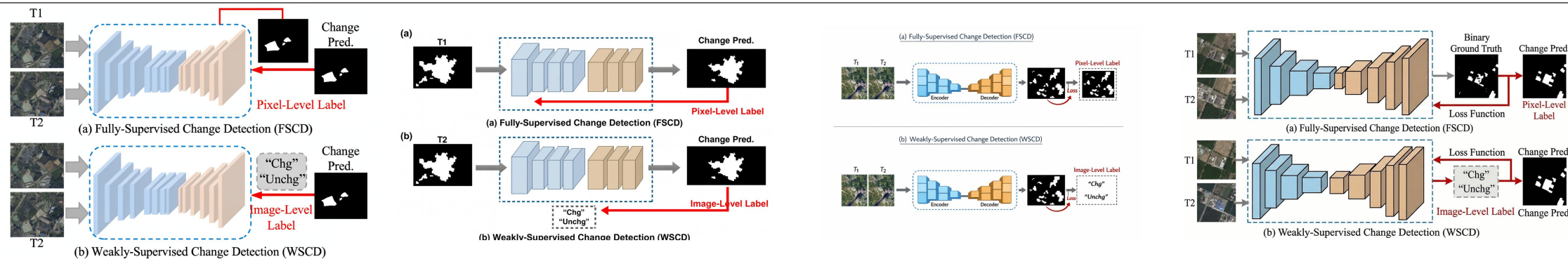
The figure illustrates a hybrid neural network architecture combining a scene feature extraction module with a modified Long Short-Term Memory (LSTM) unit, referred to as SF-LSTM (Scene-Factorized LSTM). The global layout is divided into two main parts: on the left, a scene vector extraction pipeline; on the right, the SF-LSTM cell structure enclosed within a dashed red rectangle. On the left side, an input image depicting a baseball scene is processed by a 'Places-205 CNN' model, represented as a light green rounded rectangle. The output of this CNN is fed into a '2-layer MLP' (also a light green rounded rectangle), which generates a scene vector denoted as 's', shown as a white circle. This scene vector 's' is then used to modulate the internal operations of the SF-LSTM cell. The right side of the diagram details the SF-LSTM cell. It receives inputs from two sources: the previous hidden state h_{t-1} and the current input x_t , both presented as stacked boxes. These inputs are combined and fed into three gates: the input gate 'i', the forget gate 'f', and the output gate 'o', each represented as a light green circle. Additionally, a tanh activation function is applied to the concatenated input $[h_{t-1}; x_t]$, producing a candidate cell state, which is then multiplied by the input gate 'i' via a gold-colored multiplication node (marked with an 'X'). The forget gate 'f' multiplies the previous cell state c_{t-1} (represented as a light green circle) through another gold multiplication node, and the result is added to the input gate's output to produce the new cell state c_t . The new cell state c_t is then passed through the output gate 'o' via a third gold multiplication node, and the result is activated by a tanh function (implied) to produce the new hidden state h_t , shown as a white box. Crucially, the scene vector 's' is connected via dashed blue lines to all three gates (i, f, o), indicating that it modulates or factorizes the weight matrices of these gates, as described in the caption. This factorization allows the scene context to influence the gating decisions dynamically. The internal flow of the cell state c_t is shown with a curved black arrow looping back from c_t to c_{t-1} , representing the memory retention mechanism. All connections between components are indicated by solid black arrows, except for the modulatory connections from 's' to the gates, which are dashed blue lines. The entire SF-LSTM cell is labeled with bold text 'SF-LSTM' at the top center of the dashed red boundary.



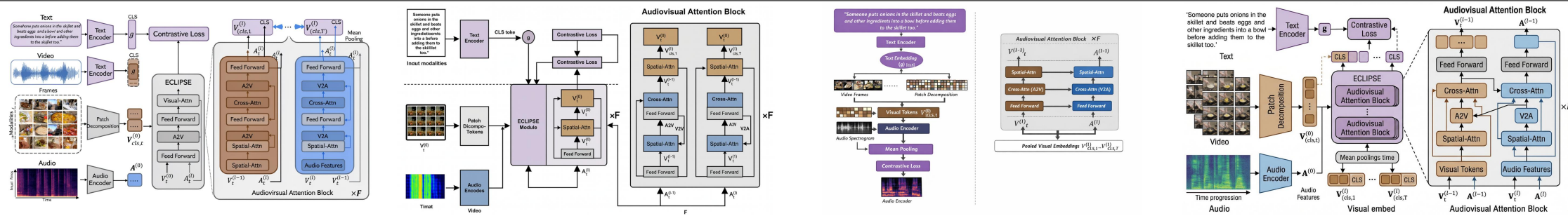
The figure illustrates a 3D-Unet architecture designed for keypoint detection in 3D medical imaging data, specifically for identifying two distinct anatomical keypoints, KP1 and KP2. The global layout follows a classic U-Net structure with an encoder-decoder architecture connected by skip connections. On the left side, two input modalities are shown: a 'one channel' input representing a single 3D volume (e.g., a bone structure), and a 'two channel' input combining this with a second modality (possibly a segmented or labeled version of the same structure). These inputs are fed into the network, which processes them through successive convolutional blocks and downsampling layers before reconstructing the output via upsampling and final convolution. The visual modules consist of orange cubes representing convolutional blocks in the encoder path, and blue cubes representing convolutional blocks in the decoder path. Each block is annotated with its feature dimension and spatial resolution: starting from 64,D,H,W at the first layer, then 128,D/2,H/2,W/2, followed by 256,D/4,H/4,W/4, and finally 512,D/8,H/8,W/8 at the bottleneck. Green downward arrows denote max pooling operations, reducing spatial dimensions at each stage of the encoder. Red upward arrows indicate conv transpose (transposed convolution) layers used for upsampling in the decoder. Gray horizontal arrows represent skip connections that concatenate feature maps from the encoder to the corresponding decoder level, preserving spatial information. A purple arrow at the end signifies a 1x1 convolution applied to the final output to produce the heatmap predictions. Connections between modules follow a clear hierarchical flow: the input passes through a series of convolutional blocks (orange cubes) with max pooling (green arrows) to progressively reduce spatial dimensions while increasing feature depth. At each downsampling stage, a skip connection (gray arrow) carries the feature map forward to the matching decoder level. In the decoder, transposed convolutions (red arrows) upsample the feature maps, which are then concatenated with the corresponding skip-connected features before being processed by further convolutional blocks (blue cubes). The final output is a pair of heatmaps, one for each keypoint (KP1 and KP2), generated via a 1x1 convolution (purple arrow) on the last decoder block. The legend on the right clarifies the meaning of each arrow type: yellow for conv block, green for max pooling, red for conv transpose, gray for skip connection + concat, and purple for 1x1 conv. The overall design enables precise localization of keypoints within complex 3D anatomical structures.



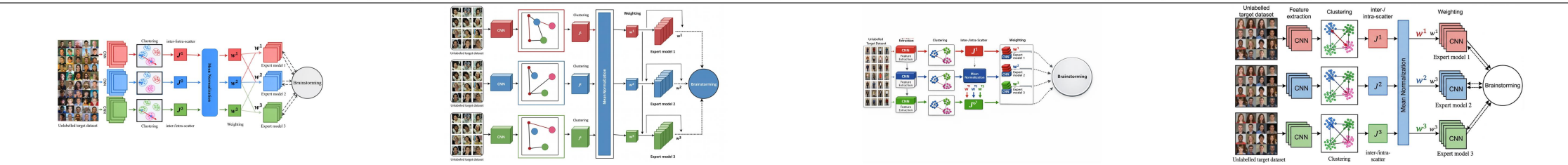
The figure presents a comparative illustration of two change detection methodologies: (a) Fully-Supervised Change Detection (FSCD) and (b) Weakly-Supervised Change Detection (WSCD), each depicted as a distinct horizontal workflow. Both workflows share a common global layout: on the left, two input satellite images labeled T1 and T2 are fed into a central deep learning model, which is enclosed within a dashed blue boundary. The model's architecture is composed of stacked 3D blocks—light blue blocks on the left representing the encoder (with decreasing block size indicating downsampling) and light brown blocks on the right representing the decoder (with increasing block size indicating upsampling). The output of the model, shown on the right, is a binary change prediction map labeled 'Change Pred.'. In part (a), FSCD, the model is trained using pixel-level supervision. A red arrow points from the 'Change Pred.' map to a 'Pixel-Level Label' map, which is a binary mask where white regions indicate detected changes. The pixel-level label is explicitly labeled in red text as 'Pixel-Level Label'. In part (b), WSCD, the same encoder-decoder model structure is used, but the supervision signal is coarser. Instead of pixel-level, the model receives image-level binary labels during training. These labels are shown as a dashed gray box containing the text 'Chg' and 'Unchg', indicating whether the entire image pair contains changes or not. A red arrow points from the 'Change Pred.' map to an 'Image-Level Label' map, which is also labeled in red text as 'Image-Level Label'. Both diagrams use gray arrows to indicate the forward data flow from inputs to predictions. The red arrows denote the feedback of supervisory signals during training. The figure emphasizes that while both methods use identical network architectures, they differ fundamentally in the type of supervision used: FSCD requires detailed pixel-wise annotations, whereas WSCD only needs simple binary labels per image pair, making it more practical for large-scale applications.



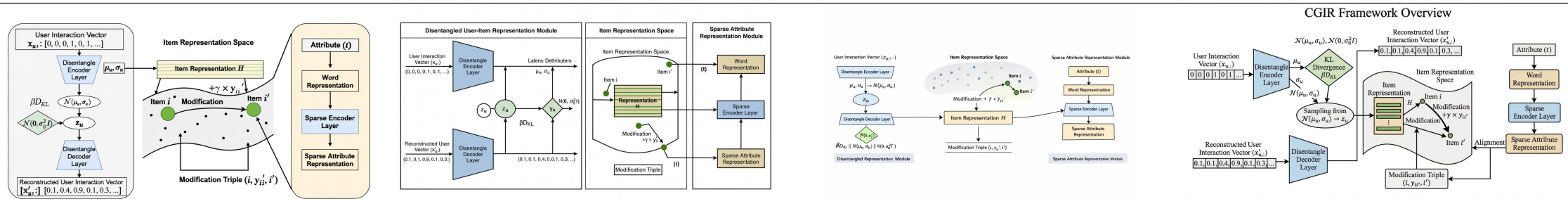
The figure illustrates the architecture of ECLIPSE, a model designed for long-range text-to-video retrieval by extending the CLIP framework with an efficient audiovisual attention mechanism. The global layout is divided into two main sections: the left side shows the overall pipeline from input modalities to the final contrastive loss, while the right side provides a detailed breakdown of the Audiovisual Attention Block, which is repeated F times. On the left, the process begins with three input modalities: text, audio, and video. The text input, represented as a sentence ('Someone puts onions in the skillet and beats eggs and other ingredients into a bowl before adding them to the skillet too.'), is processed by a Text Encoder, producing a text embedding denoted as g , which is associated with a CLS token. This embedding is used later in the contrastive loss computation. The video input consists of a sequence of frames arranged over time, shown as a grid of small images. These frames undergo Patch Decomposition, breaking them into smaller visual patches, which are then converted into visual tokens $V^{(0)}$ (cls, t). Similarly, the audio input, depicted as a spectrogram with time progression, is processed by an Audio Encoder to produce audio features $A^{(0)}$ (0). These visual and audio features are then fed into the core component, the ECLIPSE module, which is implemented as a Transformer-based backbone. The ECLIPSE module processes the visual and audio inputs through multiple stacked Audiovisual Attention Blocks. Each block contains a dual-pathway structure: one for visual tokens (brown boxes) and one for audio features (blue boxes). Each stream includes a Spatial-Attention layer (Spatial-Attn) that operates within its modality, followed by a Cross-Attention layer (Cross-Attn). The cross-attention layers enable interaction between modalities: A2V (Audio-to-Visual) allows audio to influence visual representations, and V2A (Visual-to-Audio) allows visual information to inform audio representations. Both streams also include Feed Forward layers. The inputs to each block are the outputs from the previous block, denoted as $V^{(l-1)}$ and $A^{(l-1)}$, and the outputs are $V^{(l)}$ and $A^{(l)}$. The entire block is duplicated F times, as indicated by the 'x F' notation. After processing through all F blocks, the final visual representations are aggregated via mean pooling across time, resulting in a set of visual embeddings $V^{(F)}$ (cls, t). Similarly, the audio features are aggregated via mean pooling across time, resulting in a set of audio embeddings $A^{(F)}$ (cls, t). Each associated with a CLS token. These pooled visual embeddings are then compared with the text embedding g using a Contrastive Loss, which aims to maximize the similarity between matching text-video pairs. The loss function is computed using the CLS tokens from both modalities, ensuring alignment at the global level. The figure uses distinct colors to differentiate components: purple for text-related modules (Text Encoder, Contrastive Loss, ECLIPSE), brown for visual components (Patch Decomposition, Visual Tokens, Spatial-Attn), blue for audio components (Audio Encoder, Audio Features), and gray for general processing units (Feed Forward). Arrows indicate the flow of data, with solid lines showing direct connections and dashed lines indicating conceptual or feedback links, such as ...



The figure illustrates a proposed authority regularization framework for multi-expert model learning, designed to dynamically assign weights to different expert models based on their clustering performance on an unlabelled target dataset. The global layout is a horizontal pipeline divided into three parallel processing streams on the left, converging into a central weighting mechanism, and culminating in a collaborative 'Brainstorming' phase involving three expert models on the right. Each of the three parallel streams begins with an 'Unlabelled target dataset'—a grid of 18 human images arranged in three columns and six rows. From this dataset, features are extracted using a CNN (Convolutional Neural Network) module, represented as a stack of three rectangles with the label 'CNN'. Each stream uses a distinct color: red for the top stream, blue for the middle, and green for the bottom. After feature extraction, the data undergoes 'Clustering', visualized as a square box containing three clusters of points (colored blue, green, and pink), with lines indicating intra-cluster compactness and inter-cluster separation. For instance, in the top stream, a red arrow connects two blue cluster centers, suggesting inter-cluster distance calculation. Following clustering, each stream computes 'inter-/intra-scatter' metrics, resulting in a colored square labeled J^i (blue, green, or pink). These J^i values represent the quality of clustering for each expert's features, where higher scatter indicates better discriminative power. The three J^i values are then fed into a vertical blue rectangle labeled 'Mean Normalization', which standardizes them. The output of normalization is three weight values: w^1 (red), w^2 (blue), and w^3 (green), each corresponding to one expert model. These weights are then used in a 'Weighting' step, where they are applied to the respective expert models. Each expert model is depicted as a stack of three CNNs, with the weight symbol (w^1, w^2, w^3) placed adjacent to it. The three expert models are labeled 'Expert model 1', 'Expert model 2', and 'Expert model 3', respectively. Finally, all three expert models feed into a large circular node labeled 'Brainstorming', with bidirectional dashed arrows between them, symbolizing collaborative inference or ensemble decision-making. The entire process reflects a self-supervised approach where the authority (weight) of each expert is determined by its own clustering performance on unlabeled data, aligning with the caption's explanation that larger scatter implies better discrimination capability.



The figure presents an overview of the CGIR framework, structured into three main components: a disentangled user-item representation module on the left, a sparse attribute representation module on the right, and a central alignment mechanism connecting them. The global layout is horizontal, with data flowing from left to right, and the central component acts as a bridge between the two sides. On the left, the disentangled representation module begins with a User Interaction Vector $x_{u,i}$ represented as a binary vector (e.g., $[0, 0, 0, 1, 0, 1, \dots]$). This input is processed by a Disentangle Encoder Layer, depicted as a trapezoid with a light blue fill, which outputs two parameters: μ_u and σ_u . These parameters define a Gaussian distribution $N(\mu_u, \sigma_u)$, from which a latent variable z_u is sampled. The sampling process is shown with an oval node labeled z_u . The latent variable z_u is then fed into a Disentangle Decoder Layer, which reconstructs the original interaction vector as Reconstructed User Interaction Vector $\hat{x}_{u,i}$, shown as a real-valued vector (e.g., $[0.1, 0.1, 0.4, 0.9, 0.1, 0.3, \dots]$). In the center, the Item Representation Space is visualized as a curved, wavy region containing points representing items. The latent vector z_u is connected to a block labeled 'Item Representation H ', which is a beige rectangle. This block is also connected to a block labeled 'Modification $\gamma \times y_{i,i'}$ ', indicating a modification operation based on a modification triple $(i, y_{i,i'}, i')$. On the right, the sparse attribute representation module starts with an Attribute (t) input, which feeds into a Word Representation block (beige rectangle). This is followed by a Sparse Encoder Layer (light blue rectangle), which produces a Sparse Attribute Representation. The entire framework is trained end-to-end, integrating disentanglement, sparse encoding, and representation alignment to model user-item interactions with interpretable and modifiable item representations.



Integrating disentanglement, sparse encoding, and representation alignment to model user-item interactions with interpretable and modifiable item representations.